Python

Fundamentos Essenciais

Laço for

Exceções

Funções

Lambda

Prof. Cassio Ribeiro

SENAC RJ · Tecnologia da Informação

O que vamos aprender hoje?

01

Laço for

Percorrer sequências e repetir ações

02

Exceções

Tratar erros e tornar o código robusto

03

Funções

Organizar e reutilizar blocos de código

04

Função Lambda

Funções anônimas e expressões concisas

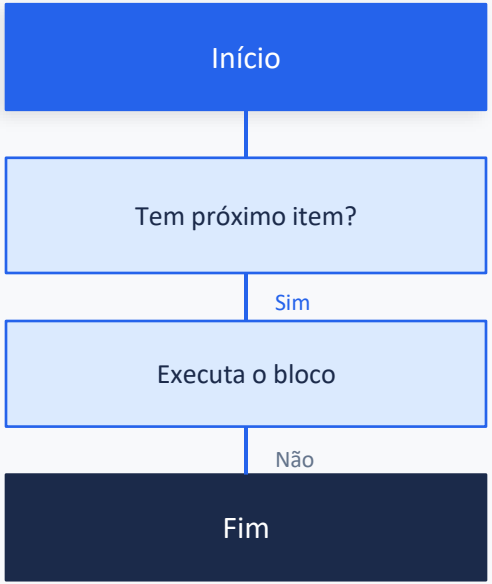
Laço for — Conceito

O que é o laço for?

O for é uma estrutura de repetição que percorre cada elemento de uma sequência — lista, string, range, tupla — executando um bloco de código para cada item.

Sintaxe básica

```
\ \ \
for variavel in sequencia:
    # bloco de código
```



Laço for — Vida Real

Onde usamos o for no dia a dia?

E-commerce

Um sistema percorre todos os produtos do carrinho de compras para calcular o total, aplicar descontos e gerar o resumo do pedido.

Relatórios Automáticos

Um gerador de relatório percorre uma lista de vendas do mês e formata cada registro para exibir no arquivo final.

Exemplo prático:

```
\\\
carrinho = {'produto': 'Notebook', 'preco': 3200},
           {'produto': 'Mouse', 'preco': 120}

total = 0
for item in carrinho:
    total += item['preco']
print(f'Total: R$ {total}')
```

Laço for — Listas, Strings e range()

Com listas

```
\\\
frutas = ['maçã', 'banana', 'uva']
for f in frutas:

    print(f)

# Saída:
# maçã
# banana
# uva
```

Com range()

```
\\\
# range(inicio, fim, passo)
for i in range(1, 6):
    print(i)

# Saída: 1 2 3 4 5

for i in range(0, 10, 2):
    print(i)  # 0 2 4 6 8
```

enumerate(), break e continue

```
\\\
nomes = ['Ana', 'Bruno', 'Carlos']
for i, nome in enumerate(nomes):
    if nome == 'Bruno':  continue  # pula Bruno

    print(f'{i} - {nome}')  # break encerra o laço antecipadamente
```

Tratamento de Exceções — Conceito

O que são exceções?

Exceções são erros que ocorrem durante a execução do programa. Sem tratamento, eles encerram o programa abruptamente. Com try/except, podemos capturar esses erros e dar uma resposta adequada.

Erros mais comuns:

`ValueError`

Tipo certo, valor errado
Ex: int('abc')

`ZeroDivisionError`

Divisão por zero
Ex: 10 / 0

`TypeError`

Tipo incompatível
Ex: '2' + 2

`IndexError`

Índice fora do range
Ex: lista[99]

Exceções — Vida Real

Onde usamos tratamento de erros?

App Bancário

Ao realizar uma transferência, o app evita que o usuário insira letras no campo de valor ou divida o saldo por zero — sem travar o aplicativo.

Sistema de Cadastro

Ao ler um arquivo CSV de usuários, o sistema captura linhas corrompidas sem interromper o processamento dos dados válidos.

Exemplo prático:

```
Saldo = 1000
try:
    valor = float(input('Digite o valor da transferência: '))
    resultado = saldo / valor
    print(f'Resultado: {resultado}')
except ValueError:
    print('Valor inválido! Digite um número.')
except ZeroDivisionError:
    print('Valor não pode ser zero!')
```

Exceções — else, finally e múltiplos except

Fluxo completo do tratamento de exceções

try	except	else	finally
Código que PODE gerar um erro	Executado SE ocorrer um erro específico	Executado SE NÃO ocorrer nenhum erro	Executado SEMPRE, com ou sem erro

```
\\\
try:
    n =
    int(input('Digite um número: '))
except ValueError:
    print('Entrada inválida!')
else:
    print(f'Número válido: {n}')
finally:
    print('Execução finalizada.')
```

Funções — Conceito

O que é uma função?

Uma função é um bloco de código nomeado que realiza uma tarefa específica. Ela pode receber dados (parâmetros), processá-los e retornar um resultado. O objetivo é evitar repetição e organizar o código.

♻️ Reutilização	📦 Organização	🔧 Manutenção	🧪 Testabilidade
Escreva uma vez, use várias vezes	Código modular e legível	Corrija em um só lugar	Teste cada parte separadamente

Sintaxe básica:

```
def nome_da_funcao(parametro1, parametro2):
    # bloco de código

return resultado
```

Funções — Vida Real

Onde usamos funções no dia a dia?

Validação de CPF

Um sistema de cadastro cria uma função validar_cpf() que é chamada em múltiplos pontos — no cadastro, na edição e na busca — sem repetir o código.

Cálculo de Frete

Uma loja virtual cria calcular_frete(peso, distancia) — basta mudar a regra dentro da função e toda a aplicação se atualiza automaticamente.

Exemplo prático:

```
\ \ \
def calcular_frete(peso, distancia, taxa=0.05):
    frete = peso * distancia * taxa
    return round(frete, 2)

# Chamadas da função
print(calcular_frete(2, 100))     # R$ 10.0 (taxa padrão)
print(calcular_frete(5, 200, 0.03)) # R$ 30.0 (taxa customizada)
```

Funções — Parâmetros padrão e *args

Parâmetro com valor padrão

Quando o argumento não for passado, o valor padrão é usado automaticamente.

```
\\\
def saudar(nome, turno='dia'):
    return f'Bom {turno}, {nome}!'

print(saudar('Ana'))            # Bom dia, Ana!
print(saudar('João', 'tarde'))   # Bom tarde, João!
```

*args — quantidade variável

Use *args quando não souber quantos argumentos serão passados.

```
def somar_tudo(*numeros):
    total = 0
    for n in numeros:
        total += n
    return total

print(somar_tudo(1, 2, 3))       # 6
print(somar_tudo(10, 20, 30, 40)) # 100
```

💡 **Dica:**

Parâmetros obrigatórios sempre vêm antes dos opcionais (com valor padrão). E *args sempre vem por último na lista de parâmetros.

Função Lambda — Conceito

O que é uma função lambda?

Lambda é uma função anônima de uma única linha. Usada para expressões simples, geralmente como argumento de outra função. Não substitui def — é complementar.

Comparação: def vs lambda

Com def:

```
def dobrar(x):
    return x * 2

print(dobrar(5))  # 10
```

Com lambda:

```
dobrar = lambda x: x * 2

print(dobrar(5))  # 10
```

```
lambda parametros : expressão          (retorno implícito)
```

Lambda — Vida Real

Onde usamos lambda no dia a dia?

Relatório de Vendas

Uma lista de produtos é ordenada pelo preço com uma única linha — sem criar uma função completa só para isso.

Filtro de Dados

Em uma análise de dados, filtramos rapidamente apenas os clientes que gastaram acima de R$ 500 usando lambda com filter().

Lambda com map():

```
produtos = [{'nome': 'TV', 'preco': 2500}, {'nome': 'Fone', 'preco': 150}]
# Ordenar por preço
ordenados = sorted(produtos, key=lambda p: p['preco’])
-----------------------------------------------------------------------------------------------
# Dobrar todos os preços com map() - map(função, lista) → transforma cada item, retorna uma nova
lista
precos = [150, 2500, 320]
dobrados = list(map(lambda x: x * 2, precos))  # [300, 5000, 640]
```

Lambda — Vida Real

Onde usamos lambda no dia a dia?

Relatório de Vendas

Uma lista de produtos é ordenada pelo preço com uma única linha — sem criar uma função completa só para isso.

Filtro de Dados

Em uma análise de dados, filtramos rapidamente apenas os clientes que gastaram acima de R$ 500 usando lambda com filter().

Lambda com filter():

```
produtos = [{'nome': 'TV', 'preco': 2500}, {'nome': 'Fone', 'preco': 150}]

# Ordenar por preço
ordenados = sorted(produtos, key=lambda p: p['preco’])
-----------------------------------------------------------------------------------------------
# Dobrar todos os preços com filter()- manter só os valores acima de 1000
precos = [150, 2500, 320]
acima1000 = list(filter(lambda x: x > 1000, precos))  # [2500]
```

Combinando tudo — Exemplo integrador

Sistema de notas escolares

for | try/except | função | lambda

```
def calcular_media(*notas):
    total = 0
    for n in notas:                          # for
        total += n
    return total / len(notas)

notas = []
while True:
    try:                                     # try/except
        entrada = input('Nota (ou Enter para finalizar): ')
        if entrada == '':
            break
        notas.append(float(entrada))
    except ValueError:
        print('Digite apenas números!')

media = calcular_media(*notas)               # função
aprovados = list(filter(lambda n: n >= 6, notas))  # lambda
print(f'Média: {media:.1f}')
print(f'Notas aprovadas: {aprovados}')
```

Exercício de Fixação

Coloque em prática!

01

Básico

Laço for + Funções

Crie uma função chamada contar_vogais(texto) que receba uma string e use um laço for para contar quantas vogais existem nela. Teste com pelo menos 3 palavras diferentes.

02

Intermediário

Exceção + Função

Crie uma função dividir(a, b) que trate os erros ZeroDivisionError e TypeError. Use try/except/else para exibir o resultado apenas quando a divisão for bem-sucedida.

03

Avançado

Lambda + map() + filter()

Dada a lista de temperaturas em Celsius [0, 15, 25, 37, 100], use map() com lambda para converter para Fahrenheit. Em seguida, use filter() com lambda para manter apenas as temperaturas acima de 80°F.

Resumo da Aula

Você aprendeu os 4 pilares essenciais de Python:

Laço for

Iterar sobre qualquer sequência de dados

Exceções

Tornar o código robusto tratando erros

Funções

Reutilizar e organizar blocos de código

Lambda

Expressões rápidas e anônimas